Hardware Accelerated Compression using Arithmetic Coding

Implementation, Measurement and Analysis

Rory Self Bryan Bong Ujjwal Uberoi Sadat Kabir

August 2026

Abstract

We accelerated lossless arithmetic coding on a Kria KV260 using four HLS designs that attack the same loop-carried dependency in different ways. This report accompanies the submitted repository. It does not restate what arithmetic coding is or re-derive the four architectures — those are covered in the presentation and in each design's `README`. It documents instead the parts that a reader of the code cannot recover from the code: the measurement method that makes the numbers comparable, the two HLS transformations that produced our best results, where the runtime actually goes, and the experiments that failed. Our fastest general-purpose design reaches 33.1 M sym/s using zero DSPs; our fastest overall reaches 286.6 M sym/s across two compute units at 12.7 nJ per byte. Every figure was measured on hardware on 8 August 2026 and each is reproducible from the repository by a documented command.

1 Scope

Four accelerators are submitted, all built, linked and measured on the board: `replication_full` (K independent coders in parallel), `interleaved` (one datapath shared C-slow across 16 coder states), `mcoder` (the H.264 CABAC M-coder, multiply-free) and `tans` (a static table-driven byte-wise coder). Each verifies itself by decoding the board's own output and comparing against the input.

This document exists because a repository cannot explain its own measurement method, and questions after our presentation made clear that *how* we measured, and *why* the results came out as they did, were the parts most worth setting out properly. Both are explained here in full.

Working through that explanation was useful in its own right. Writing down the measurement boundaries showed that one comparison in our presentation had mixed two of them, and reconstructing each figure from scratch showed that two of them could not be rebuilt from the repository as it stood. Both are now fixed, and Section 8 describes the reproduction path that resulted — which goes somewhat further than explaining the numbers, since it lets a reader regenerate them.

What follows is deliberately weighted. Material already in the presentation or recoverable from the repository is stated once and briefly; the analysis that appears nowhere else is given room. Section 2 defines the measurement method, without which none of the figures are comparable. Section 3 shows the two transformations that mattered most, at the level of the datapath rather than the source. Section 4 gives the results. Sections 5 to 7 analyse where the time and energy go, which turned out to be the most transferable finding in the project. Section 6 reports the experiments that did not work, and Section 9 the work still in progress. Appendix A reconciles these figures with those quoted in our presentation.

2 Experimental method

2.1 Two timing boundaries, never mixed

Every throughput figure in this project is measured with `std::chrono` on the ARM PS, around one of two clearly distinct regions:

Boundary	Host	Measures
kernel-only	`bench_host`	`enqueue` + `wait`. Excludes host↔device transfer
image demo	`demo_host`	the whole compress call. *Includes* that transfer

Table 1: The two measurement boundaries. Figures from different boundaries are not comparable.

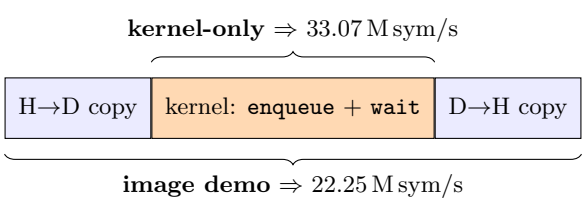

Figure 1: The two timing boundaries, drawn over one call. Both figures come from the *same* M-coder bitstream on the same input.

The distinction is not pedantic. The same M-coder bitstream measures **33.07 M sym/s** kernel-only and **22.25 M sym/s** in the image demo (Figure 1). Both are correct; they answer different questions. The kernel-only figure characterises the accelerator, the image-demo figure characterises the deployed system including its host interface. A table that mixes the two is not a comparison, so every throughput figure in this report states which boundary it uses.

2.2 Baselines

The software baseline is the same coder compiled for the ARM Cortex-A53 at `KWAY=1`, run on the board, timed identically. It is not a deliberately weakened reference. Two cautions apply:

- **The baseline must run on the A53, not the workstation.** The same source compiles to 41.9 ns/sym on x86 versus 288 ns/sym on the A53 — a factor of seven. The A53 is the correct reference because it is the processor the accelerator actually displaces.
- **Baselines differ by design and are not interchangeable.** The bit-wise arithmetic coders run at 3.47 M sym/s on the A53; the byte-wise tANS coder runs at 56.25 M sym/s, sixteen times higher. Consequently a speedup ratio is meaningless without its baseline, and *throughput*, not speedup, is the comparable quantity across designs.

For tANS specifically the baseline is an efficient byte-packing encoder, not our reference implementation, which performs a `push_back` per bit and would have inflated the reported speedup roughly threefold (18.2 rather than 56.3 M sym/s).

2.3 Correctness

Every run verifies itself: the host decodes the board's own output and compares against the input. The image demo additionally checks that the FPGA output is byte-identical to the software model, which is the stronger claim — a kernel with a subtly wrong context update would still decode its own output successfully, but would not match the model bit for bit.

2.4 Provenance tagging

Not every figure is a measurement, so each is tagged in the repository as **BOARD**, **COSIM**, **HLS**, **SW**, **DERIVED**, **PROJECTED** or **THEORY**. The distinction matters more than it sounds. A co-simulation cycle count rescaled to a design's F_{max} is a projection, not a measurement, and for `interleaved` that calculation yields anywhere between 28.3 and 35.4 M sym/s depending on which F_{max} is used — against 11.7 M sym/s actually measured on fabric.

3 Two HLS transformations, at the datapath level

Both of our fastest designs work by changing what the inner loop *is*, not by applying a pragma to the loop we started with. This section shows the two transformations, because they are the part of the work least visible from the source.

3.1 M-coder: replacing a multiply with a table lookup

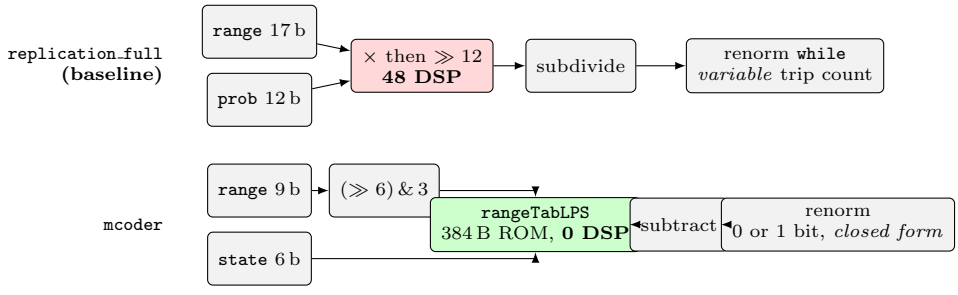

Figure 2: The M-coder transformation. Quantising the range to four buckets and the probability to a 6-bit state index turns the interval subdivision into a lookup in a 384-byte ROM, removing all 48 DSPs. The renormalisation loop, whose data-dependent trip count prevented pipelining, becomes a closed form.

In Figure 2 the multiply is the obvious cost, but the *renormalisation loop* was the one blocking II=1. In the baseline it emits a data-dependent number of bits, so HLS cannot give the loop body a fixed shape. Two properties of the M-coder remove that: the MPS path provably renormalises by either zero or one bit, and the eight-step LPS recurrence has a closed form (one barrel shift plus a prefix-AND). Both are provable rather than empirical, which is what makes the fixed-latency datapath safe. The kernel is then split into two `DATAFLOW` stages — `Bins` (bit generation) and `Pack` (byte assembly) — each achieving II=1, so the design sustains one binary decision per cycle per lane.

```
CODE_BIN(ctx, bin):                    # one binary decision, fixed latency
    state <- ctx >> 1 ; mps <- ctx & 1

    q     <- (range >> 6) & 3          # range quantile, 4 buckets
    rLPS  <- RangeTabLPS[state][q]     # 384 B ROM -- replaces range x prob
    rMPS  <- (range - rLPS) & 0x1FF

    sM    <- (rMPS >= 256) ? 0 : 1     # provably 0 or 1 bit (see text)
    sL    <- 8 - floor(log2(rLPS))     # 0..7, an 8-way priority mux

    if bin != mps:                     # LPS: upper sub-interval
        low   <- (low + range - rLPS) & 0x3FF
        range <- (rLPS << sL) & 0x1FF ;  s <- sL
        if state == 0: mps <- mps XOR 1
        state <- TransIdxLPS[state]
    else:                              # MPS: lower sub-interval
        range <- (rMPS << sM) & 0x1FF ;  s <- sM
        state <- TransIdxMPS[state]

    ctx <- (state << 1) | mps          # adaptation: one ROM read
    return RENORM(low, s)

RENORM(low, s):                        # all 8 steps classified at once
    L <- low & 0x3FF ;  p <- L[9]
    for i in 0..7 in parallel:         # unrolled, no loop-carried trip count
        cls[i] <- p ? E2 : (L[8-i] ? E3 : E1)
        p      <- p AND L[8-i]         # prefix-AND, depth-3 tree
    low <- ((L << s) & 0x1FF) | (p_at_s << 9)
    return cls[0..s-1]                 # <=8 two-bit classifications
```

Figure 3: The M-coder inner loop. Two properties make it synthesisable at II=1 the MPS renormalisation shifts by either zero or one bit, and the eight-step renormalisation recurrence has a closed form (one barrel shift plus a prefix-AND). Neither the interval split nor the model update contains a multiply — both are ROM reads — which is why the design uses zero DSPs. `RENORM` returns only a *classification* of what each step emits (E1/E2/E3), never `low` itself; that is what lets the packer run as a separate `DATAFLOW` stage.

3.2 tANS: two structural fixes, neither of them algorithmic

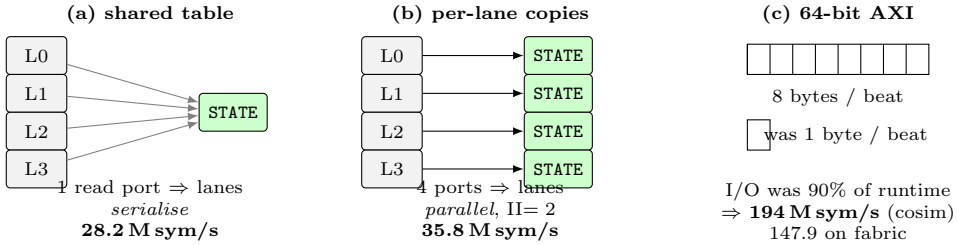

Figure 4: The tANS optimisation chain. **No algorithm changed across these three steps** — each removes a hardware blocker. The final 5.4× from widening the memory port is the single largest gain in the project.

The tANS kernel codes one whole byte per state transition, $state \leftarrow \texttt{STATE}[state][symbol]$, so there is no arithmetic in the inner loop at all. Its II is 2 rather than 1 because the table read sits inside the state recurrence — still fewer cycles per byte than the M-coder's eight binary decisions.

Figure 4 shows why the first two attempts underperformed. Replicating the coder K ways does nothing if every lane reads the same ROM: HLS gives that array one read port, so the lanes serialise and K buys almost nothing. Giving each lane a private copy of the table costs BRAM (34% per compute unit, which is what ultimately caps us at two units) but makes the lanes genuinely parallel. Only then does the memory interface become the limit, and widening it to 64 bits yields the largest single gain we measured.

The general lesson is that both designs were limited by something structural — a port, a loop shape, a bus width — rather than by the arithmetic, and that no amount of pragma tuning on the original loop would have found either fix.

4 Results

The M-coder achieves its result with **zero DSPs** by replacing the *range* × *prob* multiply with a 384-byte ROM lookup, and it compresses *5.1% better* than the coder it replaces rather than worse. Post-route timing closes with WNS +0.636 ns and no failing endpoints of 71,515.

Compression is reported separately because **ratios are only comparable on byte-identical input**, and our four designs are measured on three different workloads. On the shared 256×256 image the M-coder

Design	Throughput (M sym/s)	vs. own baseline	LUT	DSP	BRAM	F_{max} (MHz)
ARM A53, arithmetic coder (SW)	3.47	1.00×	—	—	—	—
ARM A53, M-coder (SW)	2.78	1.00×	—	—	—	—
ARM A53, M-coder (SW, real image)	2.27	1.00×	—	—	—	—
ARM A53, tANS (SW)	56.25	1.00×	—	—	—	—
`interleaved`	11.67	3.36×	23,249 (19%)	6	50 (17%)	273.97
`replication_full`	13.27	3.82×	43,573 (37%)	48	26 (9%)	273.97
`mcoder`	**33.07**	11.90×	41,797 (35%)	**0**	42 (14%)	234.47
`mcoder` (real image)	**19.33**	**8.50**×	41,797 (35%)	**0**	42 (14%)	234.47
`tans`, 1 compute unit	147.9	2.63×	19,684 (16%)	**0**	98 (34%)	224.7
`tans`, 2 compute units	**286.6**	5.10×	—	**0**	—	224.7

Table 2: All throughput figures are **kernel-only** and **BOARD** measured, 8 August 2026. Resource and F_{max} figures are **HLS** estimates for the board top. The two horizontal groups have different software baselines and their speedup columns must not be compared; compare throughput.

reaches 65,536 → 40,406 B (61.65%) and replication 65,536 → 42,661 B (65.10%), both pixel-perfect. tANS reaches 84.08% on its own workload class, which equals the source entropy: its static table is optimal for the distribution it was built from.

4.1 Sensitivity to input data

One bitstream, one host, one boundary, three images:

Image	Compressed	Ratio	FPGA
`image.pgm`	42,661 B	65.10%	10.78 MB/s
`img_smooth.pgm`	46,157 B	70.43%	10.60 MB/s
`img_noise.pgm`	66,916 B	102.11%	9.44 MB/s

Table 3: Input choice alone moves throughput by 14% and ratio from 65% to 102%. High-entropy input *expands*, as an entropy coder must.

It also settles what our benchmark inputs actually are, which matters because they are not uniform across designs. The arithmetic coders are benchmarked on `'a'+(i%7)`, a seven-symbol periodic pattern with $H = \log_2 7 = 2.81$ bits/byte. It is therefore the *most* compressible input in our set rather than a high-entropy one, and it flatters those two designs. tANS uses files drawn from its own distribution, and the demonstrations use a real image. The consequence is that throughput remains comparable across designs to within roughly 14%, but compression ratios are not comparable at all.

4.2 Getting the data is not free

Reading the 65 KB image from the board's filesystem costs **0.202 ms** warm and **2.87 ms** cold. Both the CPU and FPGA paths pay it, so it is not an offload cost — but it bounds the end-to-end result, and it bounds it hardest for the fastest designs:

Design	compress	+read	ARM +read	e2e	vs compute-only
`interleaved`	5.62 ms	5.82 ms	19.09 ms	3.28×	3.36×
`replication_full`	4.94	5.14	19.09	3.71×	3.82×
`mcoder`	3.39	3.59	29.03	8.08×	8.50×
`tans`, 1 CU	0.44	0.65	1.38	2.12×	2.63×
`tans`, 2 CU	0.229	0.431	1.38	3.17×	5.10×

Table 4: A 65,536-byte payload in a warm process, so the one-time setup of Section 5.2 is already amortised. **e2e** is (ARM+read)/(+read); **vs compute-only** is ARM/compress. The `mcoder` row is **BOARD** measured end to end on `image.pgm` (kernel 3,390 μs, its own ARM coder 28,826 μs); the other rows are **DERIVED** from the kernel-only throughputs of Table 2 and share its synthetic `'a'+(i%7)` workload and its 3.47 M sym/s ARM baseline. The 0.202 ms file read is **BOARD** measured. tANS rows are illustrative: 65 KB is not its native 16 KB block and its ARM baseline differs.

The trend is monotonic and is the point: the faster the accelerator, the more of its advantage the unaccelerated read consumes. `interleaved` loses 8% of its speedup, `mcoder` 25%, and `tans` at two compute units loses **62%** — from 5.10× to 1.96× — because 0.75 ms of file I/O is more than three times its entire 0.23 ms of compute. Amdahl's law arrives at the file system, and it arrives first for the design that was most

worth accelerating. Reading the file faster, or overlapping it with compute, would be worth more to `tans` than another compute unit.

5 Where the runtime goes

The bottleneck in this project is not one thing, and it moved as the designs improved. The measurements below locate it: in three of the four designs the coder is no longer the limiting factor at all, and what replaced it is data movement.

Design	Data movement is…	Evidence
`replication_full`	negligible; bursts infer cleanly	cosim 12.9 vs. fabric 13.27 M sym/s (3%)
`interleaved`	∼15% of latency, bursts *revert*	1.46× DDR penalty, 198 µs → 289 µs
`mcoder`	**84% of runtime**	coder is 4,096 of 26,317 cycles
`mcoder` (16 KB)	**84% → 60% genuine movement**	fixed-cost share 44% → 16% (N-sweep fit)
`tans`	**90% of runtime**, before widening	35.8 → 194 M sym/s from a wider port

Table 5: The bottleneck migrated out of the coder and into the memory interface.

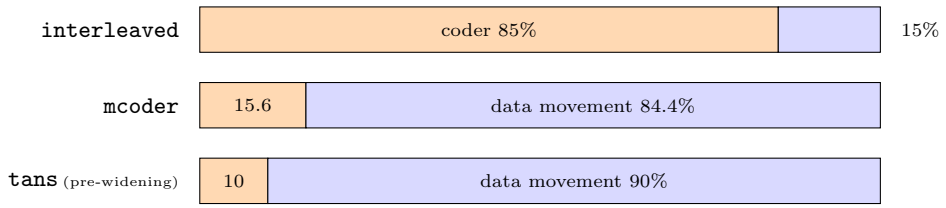

Figure 5: Share of runtime spent coding versus moving bytes. The bottleneck migrated out of the coder entirely in two of the three designs shown. `replication_full` is omitted because its split was never separately quantified — its co-simulation and fabric figures agree to 3%, which bounds its data-movement cost as small but does not measure it.

For the M-coder this is decisive (Figure 5). At K=8 the coder occupies 4,096 cycles of a 26,317-cycle call — **15.6%**. The remaining 84% is byte-wide `m_axi` copy loops which move the same total bytes *regardless of* K. This single fact explains three results that otherwise look contradictory: the engine became roughly ten times faster (83.7 → 8 cycles/byte single-stream) while the system gained only 2.49×; K=16 fits after the LUTRAM change yet runs *slower* than K=8; and returns collapse well before K=16 (K=1 → 2 gains 36%, 4 → 8 gains 7%, 8 → 16 is negative). It is also why the replication scaling is sub-linear rather than linear as presented. Post-route analysis confirms it independently: the critical path lies in the output copy loop, not the coder.

This 84% figure is a single point on the same block-size curve as Section 5.1: converting the fitted $t = t_{\text{fixed}} + Nt_{\text{sym}}$ into cycles at N=4095 reproduces 26,317 exactly. The sweep lets that 84% be split further than one measurement can: of it, 44 points are XRT launch and kernel pipeline-fill — fixed, independent of N — and only 40 points are the AXI `Split`/`Concat` copy loops actually moving bytes (2.60 cycles/byte, back-solved from t_{sym} minus the coder's known 1.00 cycle/byte). Rebuilding at N=16384 confirms the split: the per-byte copy rate is unchanged (2.57 cycles/byte, within noise), but the fixed share collapses from 44% to 16%, so the honest data-movement bottleneck — now 60% of the call — finally dominates measured throughput (31.2 → 46.8 M sym/s) the way the original 84% figure implied it already did.

5.1 Host interface overhead

Fitting per-call latency as $t = t_{\text{fixed}} + N \cdot t_{\text{sym}}$ over block sizes $N \in \{256, 1024, 2048, 4095\}$ separates the two contributions:

Design	t_{fixed}	t_{sym}	Asymptote	Block
`replication_full`	55 µs	62.2 ns	16.1 M sym/s	4 KB
`interleaved`	62 µs	70.5 ns	14.2 M sym/s	4 KB
`mcoder` (4 KB build)	57.9 µs	18.00 ns	55.6 M sym/s	4 KB
`mcoder` (16 KB build)	56.8 µs	**17.85 ns**	**56.0 M sym/s**	16 KB
`tans`	∼24 µs	—	—	16 KB

At N=4095 the fixed XRT cost is 18% of a call; at N=256 it is 77% (Figure 6). Every throughput figure we report is therefore quoted at the largest block, because a small-block measurement characterises XRT rather than the design.

For `interleaved` the cosim-to-fabric gap decomposes cleanly. Both run at 200 MHz, so there is no clock term: the measured 351.0 µs per call minus the independently-fitted 62 µs overhead leaves 289 µs of kernel

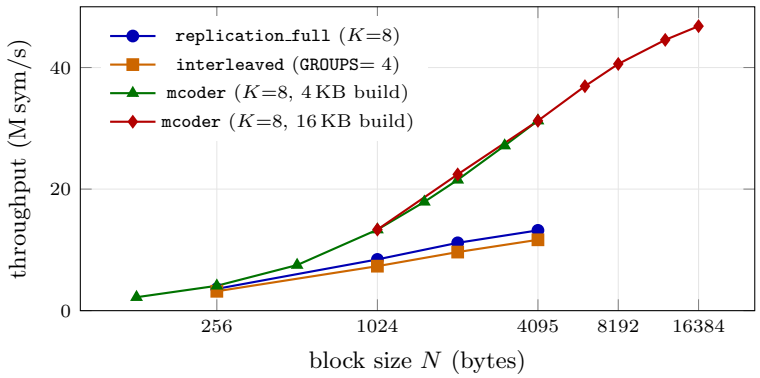

Figure 6: Throughput against block size, kernel-only, measured on the board. Every `mcoder` point was verified lossless. The two `mcoder` curves are separate bitstreams differing only in `MC_MAX_IN`; they coincide where they overlap. The 4 KB designs are still climbing steeply at their N=4095 ceiling, which is the point: what they report is dominated by a fixed per-call cost they never amortise.

time against co-simulation's 198.1 μs. The residual 1.46× is real DDR behaviour that co-simulation cannot show, since it models AXI as a fixed 64-cycle latency and hides non-coalesced access entirely. We present this as a decomposition rather than a validated model: the 1.46× is defined as the leftover ratio, so it cannot fail to reconcile. What it establishes is that host overhead and memory behaviour together account for the gap, with no third term required.

5.2 Payload size: when is offloading worth it?

Section 5.1 varies the size of one kernel call. This varies the total amount of data compressed, split across $\lceil total/block \rceil$ successive calls — the way a real caller must, and the way our image demo already chunks a picture. The two are independent axes and they answer different questions.

Payload	Calls	Kernel only	+DMA	+setup
4 KB	1	31.4	29.5	0.13
16 KB	1	47.3	45.6	0.54
64 KB	1	53.3	52.2	2.10
256 KB	4	53.4	52.2	7.55
1 MB	16	53.3	51.7	20.95
4 MB	64	53.3	51.5	37.63
16 MB	256	53.3	51.3	46.98

Table 6: `mcoder` at a 64 KB block, all figures M sym/s, **BOARD** measured, every configuration verified lossless. The kernel column is flat once the payload fills one block; the end-to-end column is the same hardware doing the same work, with a one-time ∼30 ms setup amortised over a growing payload.

Three things follow, and the first is a negative result worth stating plainly.

Total payload size does not affect throughput. Across a 4096× range of payload the kernel-only column moves by under 5% (and the same holds at a 4 KB block, flat at ∼31 M sym/s). Total time is simply (number of calls) × (per-call time), with no term that grows with payload — no memory-system degradation even at 4,096 consecutive calls. This is what licenses using the per-call model of Section 5.1 to predict anything at all, and it means *payload size is the wrong knob*: scaling the data 4096× changes throughput by under 5%, while scaling the block 16× changes it by 1.72×.

Including setup, there is a minimum payload below which offloading loses. The +setup column is what a process that starts, compresses once and exits actually experiences. Against `mcoder`'s own ARM baseline of 3.20 M sym/s, break-even is at ∼**100 KB** (64 KB block) or ∼107 KB (4 KB block).

Our own image demo sits below that line. At 65,536 B the end-to-end figure is 2.10 M sym/s against the CPU's 3.20 — the FPGA is **1.52× slower** for a single image on a cold process. The 65 KB compresses in 1.23 ms; the process spends 29.8 ms getting ready to do it. This does not contradict the 9.53× of Table 2, which is kernel-only and correct for characterising the accelerator. The accelerator wins from ∼100 KB in a single process, or immediately in any process that stays alive and amortises setup across payloads — which is the realistic deployment, and is exactly what the flat kernel column describes.

We note the ∼30 ms setup is a *warm* xclbin load; the first load after `xmutil loadapp` measures 217 ms, which moves break-even out to ∼723 KB. Both are real; which applies depends on whether the bitstream is already resident.

6 Experiments that did not work

These shaped the final designs and are reported because they are as informative as the successes.

Wide AXI helps one design and harms another. Widening the memory port gave tANS a **5.4**× speedup (35.8 → 194 M sym/s), the single largest gain in the project — larger than adding a second compute unit. The identical change made `interleaved` **32.9% slower** (39,623 → 52,635 cycles). tANS reads one aligned 64-bit word after another; `interleaved` codes K contiguous chunks and must gather an *unaligned* word at every chunk boundary, where realignment costs more than widening saves. Wide AXI is therefore not a general optimisation: it pays when access is aligned and sequential. Fix the access pattern before widening the port.

Streaming (DATAFLOW) is blocked by the algorithm, not the tools. On tANS, DATAFLOW measured 25,461 cycles against 16,851 without — 50% *slower*. tANS encodes in reverse, so the coder cannot begin until the whole block has arrived; Load can never overlap the encoder and the stream handshaking is pure added cost. This is a structural property of ANS. On `interleaved` the ceiling is different but equally limiting: data movement is only ∼15% of latency there, so perfect overlap could not exceed 1.18×, which does not justify the restructure.

An F_{max} above the clock target does not survive place and route. One variant (`GROUPS=2` with a registered DSP multiply) reported an HLS F_{max} of 402.58 MHz at II=1. Linking it against the platform's 400 MHz clock failed: `[VPL 101-2] design did not meet timing --- pulse width violation`. A 402 MHz estimate against a 400 MHz target is a 0.6% margin, well inside the error of a model that accounts for logic delay but not placement, routing congestion or clock skew. The general lesson, which the platform's quantised 100/200/400 MHz clocks reinforce, is to design to the clock you will actually receive rather than to F_{max}: our 273.97 MHz designs run at 200 MHz and the headroom is simply discarded.

Maximum-lane replication does not fit. A reduced-model variant (K=31, eight probabilities instead of the 255-node context tree) reaches 28.20 M sym/s in co-simulation at 95% LUT. It was never built: 95% LUT before the platform's own logic is beyond the practical placement ceiling, and it is dominated anyway by `interleaved`, which reaches a similar throughput class at 19% LUT while keeping the full model. It stands only as a theoretical upper bound.

Pipelining the exact coder does not converge if the multiply cannot be bounded. Before either successor design existed, iteration 2 tried to pipeline V5's single-stream coder directly: a flat state machine ($S_{\text{CODE}}/S_{\text{RENORM}}/S_{\text{DRAIN}}/S_{\text{FLUSH}}$, one fixed-shape unit of work per cycle) under `#pragma HLS PIPELINE II=1`, replacing V5's variable-length renorm loop that HLS could not schedule at all. It worked functionally — bit-exact with V5 across $K = 1..16$, lossless round-trip, and a software work-unit count predicting 11.7–21.1 cycles/symbol against V5's measured 83.7, a 4–7× single-stream gain on top of replication. It never closed timing. `low`, `high` and `prob` are loop-carried across the FSM's state merges, so Vitis's width optimiser could not prove their bounds and inferred a full 32×32 multiply — three-plus cascaded DSPs on the critical path — instead of the intended 17×12. Explicit mask hints on both operands, added after the first synthesis attempt, did not change the inference. The design was abandoned unsynthesised, but the lesson carried forward directly: the M-coder's own masks on `low`/`range` succeed at the identical job only because their bounds are provable by construction and its ROM lookup sidesteps the problem outright by removing the multiply rather than trying to narrow it.

7 Energy

Energy cannot be obtained from a synthesis report: Vitis reports resources and timing only, and Vivado's `report_power` is a switching-model estimate. These figures are read from the KV260 SOM's INA260 sensor (`/sys/class/hwmon/hwmon0/power1_input`) sampled at approximately 5 Hz while each workload runs continuously.

Three observations follow. First, **energy per byte is set by throughput, not by power**: board power varies by only 9% across everything measured while throughput varies by a factor of 82, so the fastest design is essentially always the most energy-efficient — race to idle dominates. The FPGA draws *more* instantaneous power than the CPU yet uses far less energy per byte. Reporting watts alone would have made the accelerator appear worse.

Second, **a second compute unit is the best energy result available**, nearly halving energy per byte for roughly 5% more power: parallelism at constant clock is close to free energetically.

Workload	Throughput	Board power	Energy/byte
ARM software, arithmetic coder	3.46 M sym/s	∼3.36 W*	∼971 nJ/B
FPGA `interleaved`	11.61 M sym/s	3.362 W	290 nJ/B
FPGA `replication_full`	13.23 M sym/s	3.318 W	251 nJ/B
ARM software, tANS	54.35 M sym/s	3.361 W	61.8 nJ/B
FPGA `tans`, 1 CU	145.7 M sym/s	3.443 W	23.6 nJ/B
FPGA `tans`, 2 CUs	**286.6 M sym/s**	3.634 W	**12.7 nJ/B**

Table 7: *This row's power was not measured directly and is estimated from a saturated A53 core; every other row is measured. Idle floor 3.22–3.29 W.

Third, **interleaving costs energy relative to replication** (290 versus 251 nJ/B). It is both slightly slower on this board and draws slightly more power, because it trades DSPs for BRAM (50 versus 26 blocks) and BRAM accessed every cycle is not cheap. Its area advantage does not translate into an energy advantage.

We flag one caveat: the idle floor drifts by around 70 mW between sessions, comparable to the smallest incremental signal here, so total-power figures are robust but incremental-power figures should be treated as indicative.

8 Reproducing these results

The repository documents this in full, so it is stated here only briefly. Each design directory carries a `REPRODUCIBILITY.md` giving, for every number claimed, the file, the command, the machine to run it on and the expected output; raw logs sit alongside. `run_on_board.sh` deploys any design, loads the bitstream, runs it and checks losslessness, and `env.sh` locates the toolchain on an arbitrary machine.

Two defects had to be repaired first, both of which are worth recording because neither produced any error until someone tried to reproduce the work. The **M-coder bitstream had never been committed**, so the design behind our best general-purpose result could not be run from the repository at all; it has been rebuilt from the committed `.xo`, closes timing at WNS +0.636 ns, and emits byte-identical output (1,687 B) to the original record. Separately, **co-simulation could not compile its own testbench** in two designs, whose `tb.cflags` lacked `-I../src` while their testbenches include a header from that directory. Because `tb.file` does not add an include path, co-simulation failed before starting — so every co-simulation figure we had published for those designs was unreproducible as committed. Both now re-run and reproduce their published values exactly.

9 Work in progress

Three experiments are in progress and are not yet reflected in the results above. We state them here rather than omit them, since each would change how the results should be read.

1. Behaviour across block size, from small to large. Figure 6 already shows this for two designs over $N \in \{256 \ldots 4095\}$, and the behaviour is not yet asymptotic — both curves are still climbing at the largest block we measured. Completing this means extending the sweep to `mcoder` and `tans`, and pushing N well past 4 KB until throughput flattens. We expect the four designs to separate differently at small N, where the fixed per-call cost dominates (77% of a call at N=256), than at large N, where the memory interface should dominate. That crossover is the interesting result and we do not currently have it.

2. A common corpus across all implementations. Our designs are currently benchmarked on three different workloads, which is the single largest methodological weakness remaining. Throughput is comparable to within about 14% regardless (Table 2), but compression ratios are not comparable at all. The fix is to run every design over one identical corpus. Note that tANS cannot simply join that corpus without defeating its premise: its table is precomputed from one distribution, so it must additionally be reported on its own class, with the difference between the two figures being the honest cost of a static model.

3. A multi-threaded software baseline. Every speedup in this report is against a *single-threaded* A53. The KV260 has four cores, and the workload parallelises across independent chunks exactly as our hardware does, so a fair software baseline should use all of them. We expect this to reduce every reported speedup by up to roughly a factor of four — our 9.53× would become approximately 2.4× against a fully parallel baseline. We consider this the correct comparison to publish, and note that it does not affect the energy results, where the FPGA advantage comes from finishing sooner at similar power and a busy four-core CPU would draw substantially more.

10 Limitations

We state these plainly rather than leaving them to be discovered. They are distinct from Section 9, which is work in progress rather than a limit of the approach.

- **Encoder only.** No hardware decoder was attempted, so we have not demonstrated a complete codec in fabric.
- **Order-0 model.** The largest available compression gain remains unbuilt; chunking additionally costs about nine percentage points (37.4% → 46.50%) because each chunk restarts the model.
- **Multiple compute units were only applied to tANS.** The measured 1.95× was never carried to the other designs; the M-coder in particular is 84% data movement and never received a wider port, which we believe is the largest remaining gain available.
- **Static table for tANS.** It is lossless on any input but only compresses data matching its baked distribution, expanding an ELF binary to 143%. This is the design's premise rather than a defect, but it bounds where the result applies.

A Reconciliation with the figures quoted in our presentation

Some figures here differ from those given in our presentation, mostly because this report states the measurement boundary explicitly where the talk did not. They are listed so the two can be read side by side.

Presented	Correct	Cause
M-coder 21.16 M sym/s	**33.07 M sym/s**	21.16 is an image-demo figure; the other three bars were kernel-only
6.09× the ARM baseline	**9.53**×	image-demo numerator over a kernel-only denominator
"Replication scales… Linear"	strongly **sub-linear**: 4.05× at K=8, 4.49× at K=16	lane count does not divide the data-movement cost (Sec. 5)
"Storing contexts in **BRAM** is the better area trade"	**LUTRAM** (distributed RAM); BRAM is unchanged at 42 blocks either way	the build flag is `-DMC_CTX_LUTRAM`; only LUT usage moves
83,156 LUT at K=16	**83,337**	transcription
Sweep table column "K=8" (fourth)	K=16	label error
"predicted 31 M sym/s"	a **projection**; co-simulation gives 20.68	31 is the cosim cycle count rescaled to F_{max}
"a 293 MHz design"	**273.97 MHz** (board top)	two different synthesis tops; both figures were real

Table 8: Figures restated against an explicit measurement boundary. Each either improves the result or makes its status precise.

The first two share one cause: the M-coder bar came from the image demo while the other three came from the kernel-only path (Section 2). Both numbers are correct for what they measure; read on a single axis the M-coder is the stronger result, 9.53× rather than 6.09×.

The F_{max} entry is not an error in either figure. `synth/hls.cfg` synthesises the coder alone and reports 342.47 MHz; `synth/board.cfg` synthesises the coder plus its AXI wrapper — the design that is actually linked — and reports 273.97 MHz. Both numbers had appeared in our notes without their context. The 20% difference between them is itself a result: the critical path of the built design lies in the memory interface, not in the coder.

References

[1] I. H. Witten, R. M. Neal, and J. G. Cleary, "Arithmetic coding for data compression," *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, 1987.

[2] D. Marpe and T. Wiegand, "A highly efficient multiplication-free binary arithmetic coder and its application in video coding," *Proc. IEEE ICIP*, Barcelona, 2003, pp. 263–266.

[3] D. Marpe, H. Schwarz, and T. Wiegand, "Context-based adaptive binary arithmetic coding in the H.264/AVC video compression standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 7, pp. 620–636, 2003.

[4] J. Duda, "Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding," arXiv:1311.2540, 2013.

[5] P. G. Howard and J. S. Vitter, "Practical implementations of arithmetic coding," in *Image and Text Compression*, Kluwer, 1992, pp. 85–112.

[6] C. E. Leiserson and J. B. Saxe, "Retiming synchronous circuitry," *Algorithmica*, vol. 6, pp. 5–35, 1991.